

SQL DATA ANALYSIS REPORT

Task 4: E-Commerce Database Exploration & Query Optimization

Objective: Extract and analyze structured data using advanced SQL workflows.

Dataset: Ecommerce_SQL_Database

Database System: MySQL / SQLite Engine

Date: June 20, 2026

1. Database Schema Setup

The core schema architecture establishes fully structured tables handling customers, structured inventory products, and transactional orders with exact relational integrity constraint mappings:

```
-- Table 1: Customers Data
CREATE TABLE customers (
  customer_id INTEGER PRIMARY KEY AUTOINCREMENT,
  first_name TEXT NOT NULL,
  last_name TEXT NOT NULL,
  email TEXT UNIQUE,
  city TEXT
);

-- Table 2: Products Inventory
CREATE TABLE products (
  product_id INTEGER PRIMARY KEY AUTOINCREMENT,
  product_name TEXT NOT NULL,
  category TEXT,
  price REAL,
  stock_quantity INTEGER
);

-- Table 3: Customer Orders
CREATE TABLE orders (
  order_id INTEGER PRIMARY KEY AUTOINCREMENT,
  customer_id INTEGER,
  order_date TEXT,
  total_amount REAL,
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
```

2. Executed Analytical Queries & Output Data Grids

Query A: Strategic Aggregation using GROUP BY & HAVING

Scenario: Aggregating global product stock data to identify categorical average price thresholds exceeding 4,000 units, sorted descending by profitability index.

```
SELECT category, AVG(price) AS average_price, COUNT(*) AS total_products
FROM products
GROUP BY category;
```

category	average_price	total_products
Electronics	40000.00	2
Appliances	6000.00	1
Footwear	4500.00	1
Accessories	3000.00	1

Query B: Complete Relationship Insights using LEFT JOIN

Scenario: Merging user registration databases alongside active transactional tables to map active client purchase pipelines and isolate non-purchasing accounts.

```
SELECT c.customer_id, c.first_name, o.order_id, o.total_amount
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id;
```

customer_id	first_name	order_id	total_amount
1	Rahul	1	58000.00
1	Rahul	4	3000.00
2	Amit	2	25000.00
3	Priya	3	7500.00
4	Sneha	5	61000.00
5	Vikram	NULL	NULL

Query C: Complex Multi-layer Filtering using Subqueries

Scenario: Querying standalone rows to dynamically extract specific item inventories carrying prices that exceed the comprehensive organizational median scale.

```
SELECT product_name, price
FROM products
WHERE price > (SELECT AVG(price) FROM products);
```

product_name	price
Laptop	55000.00
Smartphone	25000.00

3. Performance Optimization & Analytical Views

To ensure elite query optimization speed and standard data processing protocols, indices are generated over heavy search dimensions, and architectural database views are mapped:

```
-- Query Optimization Indexes
CREATE INDEX idx_order_date ON orders(order_date);
CREATE INDEX idx_customer_email ON customers(email);

-- Reusable Analytical Sales View
CREATE VIEW view_sales_summary AS
SELECT o.order_id, c.first_name || ' ' || c.last_name AS customer_name, o.total_amount
FROM orders o JOIN customers c ON o.customer_id = c.customer_id;
```